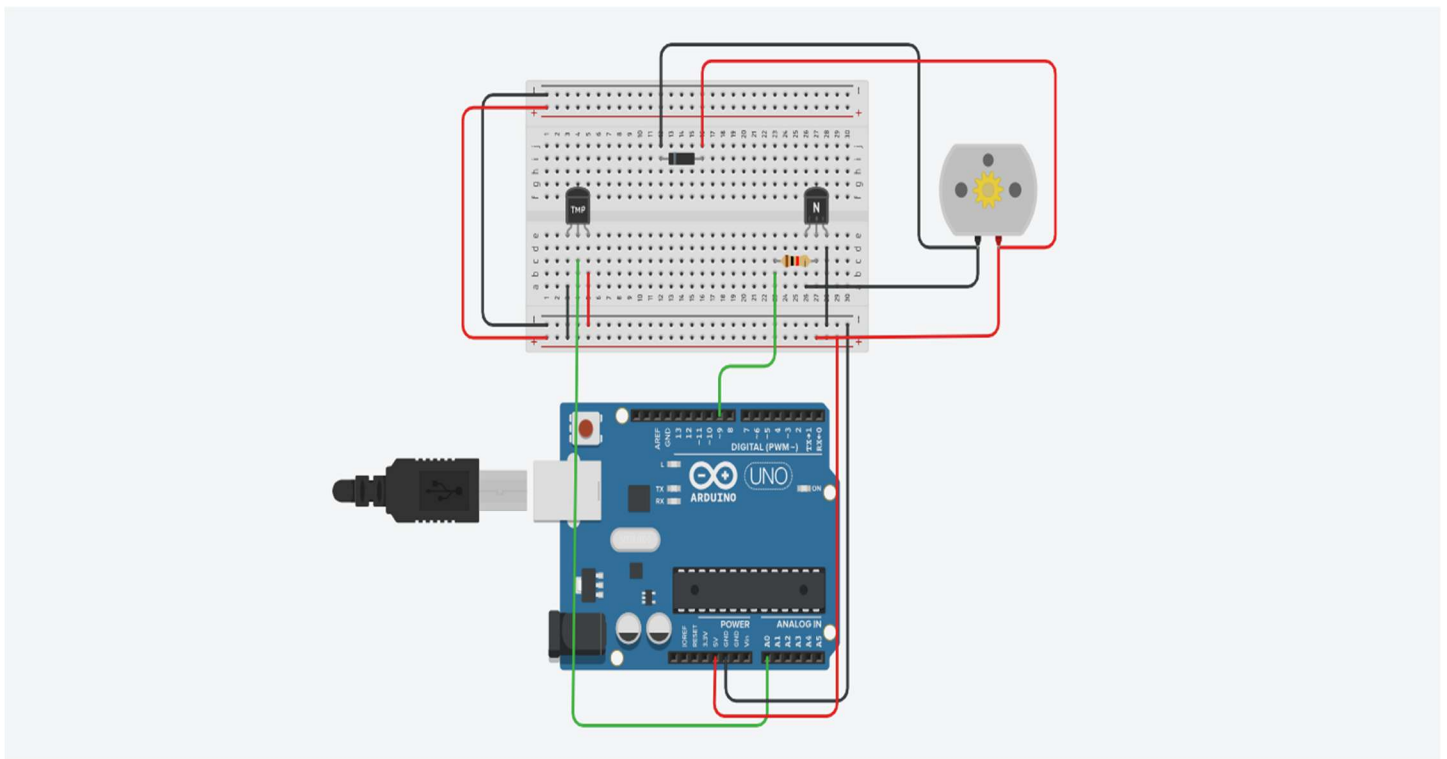


PROJECT REPORT

Automatic Fan Speed Control Using Temperature Sensor

An Embedded Systems Project Based on Arduino UNO and TMP36 Temperature Sensor



Designed By

Galla Indranag

B.Tech ECE, NRI Institute of Technology, Guntur

Embedded Systems and IoT Engineering (ESSCI ELE/Q1404), SRM University AP

1. Abstract

This project implements an automatic fan speed control system using an Arduino UNO microcontroller and a TMP36 analog temperature sensor. The system continuously monitors ambient temperature and adjusts the speed of a DC fan proportionally using Pulse Width Modulation (PWM), eliminating the need for manual intervention. As temperature rises, the fan speed increases in defined steps, providing an energy-efficient, low-cost cooling solution suitable for embedded applications such as electronic enclosure cooling, home automation, and industrial thermal management.

Traditional cooling systems typically rely on simple ON/OFF thermostatic switches, which cause abrupt starts and stops, mechanical wear, and inefficient power usage. By contrast, the system described in this report uses a microcontroller-based closed-loop decision process: the analog voltage from the TMP36 sensor is periodically sampled by the Arduino's onboard 10-bit Analog-to-Digital Converter (ADC), converted into a temperature reading in degrees Celsius, and mapped to one of three discrete PWM duty cycles. This duty cycle is then applied to a DC fan through a transistor switching stage, giving the fan three distinct operating speeds OFF, medium, and full depending on the sensed thermal condition. The complete design, including schematic, firmware, and simulated test results obtained using Autodesk Tinkercad, is presented in the sections that follow.

The project was undertaken as part of the Embedded Systems and IoT Engineering curriculum (ESSCI ELE/Q1404) and demonstrates the practical integration of analog sensing, embedded C programming, PWM-based actuation, and transistor-level power switching all fundamental competencies for an embedded systems engineer.

2. Introduction

Temperature regulation is one of the most common requirements in both consumer electronics and industrial systems. Overheating can degrade the performance and lifespan of electronic components, motors, and processors, making active cooling an essential design consideration. Conventional cooling solutions often run fans at a fixed speed regardless of actual thermal load, which wastes energy, generates unnecessary noise, and accelerates component wear.

Embedded systems provide an elegant solution to this problem by allowing continuous, intelligent monitoring of environmental parameters and automated response through actuators. This project uses the Arduino UNO, a widely used open-source microcontroller development board built around the ATmega328P chip, together with the TMP36 analog temperature sensor, to build a low-cost, easily reproducible automatic fan speed controller. The Arduino platform was selected for its simplicity, extensive community support, low cost, and suitability for rapid prototyping making it ideal for an academic embedded systems project of this scope.

The system is designed around three operating zones: a low-temperature zone in which no cooling is required and the fan remains off to conserve power and reduce noise; a medium-temperature zone in which the fan runs at approximately half speed to provide moderate cooling; and a high-temperature zone in which the fan runs at full speed to provide maximum cooling. This tiered approach mimics the behaviour of more expensive commercial fan controllers while using only a handful of low-cost components.

3. Objective

- To design a temperature-sensing circuit using the TMP36 sensor interfaced with Arduino UNO.
- To convert the analog temperature reading into a corresponding PWM signal using embedded C logic.
- To drive a DC fan through an NPN transistor switch controlled by a PWM output pin.
- To protect the switching transistor from inductive voltage spikes using a flyback diode.
- To display real-time temperature and fan PWM values on the Serial Monitor for verification and debugging.

- To validate the system's response across low, medium, and high temperature ranges through simulation in Tinkercad.
- To document the complete design process, from theory to testing, as a reference for future enhancement (e.g., IoT-based monitoring).

4. Literature Survey

Automatic temperature-based fan control is a well-established application area within embedded systems and has been implemented using a variety of approaches in academic and commercial contexts:

- Bimetallic thermostat fans: Mechanical devices that use a bimetallic strip to physically open or close a switch at a fixed temperature threshold. These are simple and cheap but offer only ON/OFF control with no intermediate speeds and are prone to mechanical wear.
- Relay-based microcontroller systems: Use a microcontroller with a digital temperature sensor and a relay to switch the fan ON/OFF. These improve on bimetallic thermostats by allowing configurable thresholds but still lack proportional speed control.
- PWM-based analog/digital sensor systems (this project's approach): Use PWM to vary fan speed proportionally to temperature, offering smoother transitions, reduced noise, and better energy efficiency than binary ON/OFF control.
- IoT-integrated systems: Extend PWM-based control with wireless connectivity (e.g., ESP32, Wi-Fi/MQTT) to allow remote monitoring and logging of temperature and fan status an identified direction for future enhancement of this project.

This project adopts the PWM-based proportional control approach, striking a balance between implementation complexity and functional improvement over simple ON/OFF systems, making it well suited to an introductory embedded systems course project.

5. System Requirements

5.1 Hardware Requirements

- Arduino UNO R3 development board
- TMP36 analog temperature sensor
- NPN transistor (2N2222 type or equivalent)
- DC fan / motor (5V rated, simulated in Tinkercad)
- 1N4001 (or equivalent) flyback diode
- 1k Ω current-limiting resistor
- Breadboard and jumper wires
- USB cable for programming and power

5.2 Software Requirements

- Arduino IDE
- Embedded C / Arduino programming language
- Autodesk Tinkercad used for circuit simulation and validation prior to physical hardware testing

6. Components Used and Theory

This section describes each hardware component used in the project along with its role and relevant electrical characteristics.

6.1 Arduino UNO R3

The Arduino UNO is an open-source microcontroller board based on the ATmega328P 8-bit AVR microcontroller. It operates at 5V logic level, has 14 digital I/O pins (6 of which support PWM output), 6 analog input pins connected to a 10-bit ADC, 32KB of flash memory, 2KB of SRAM, and 1KB of EEPROM. In this project, the Arduino serves as the central processing unit: it reads the analog voltage from the TMP36 sensor, computes the temperature, applies the control logic, and generates the PWM signal that drives the fan.

6.2 TMP36 Temperature Sensor

The TMP36 is a low-voltage, precision analog temperature sensor manufactured by Analog Devices. It provides an output voltage that is linearly proportional to temperature in Celsius, with a typical accuracy of $\pm 1^\circ\text{C}$ and an output scale factor of 10mV per $^\circ\text{C}$. It has a 500mV offset at 0°C , allowing it to measure negative temperatures without requiring a negative supply voltage. The sensor has three pins: Vcc (2.7V - 5.5V), Vout (analog output), and GND. In this project it is powered from the Arduino's 5V rail and its output is connected to analog pin A0.

6.3 NPN Transistor (Switching Element)

An NPN bipolar junction transistor (BJT), such as the 2N2222, is used as an electronic switch to drive the DC fan. Since the Arduino's digital output pins cannot supply the current required by a motor directly, the transistor is used in switching mode: a small base current (controlled via a current-limiting resistor and the PWM signal from pin 9) controls a much larger collector-emitter current flowing through the fan. This allows the low-power microcontroller output to control a higher-power load safely.

6.4 Flyback (Freewheeling) Diode

DC motors and fans are inductive loads. When the current through an inductive load is suddenly interrupted (as happens continuously during PWM switching), the collapsing magnetic field induces a large reverse voltage spike that can damage the switching transistor. A flyback diode is connected in parallel with the fan, oriented to conduct only during the OFF phase of switching, providing a safe path for this induced current and protecting the transistor from damage.

6.5 Current-Limiting Resistor

A resistor (approximately $1\text{k}\Omega$) is placed in series with the transistor's base terminal to limit the base current to a safe value, preventing damage to both the transistor and the Arduino's output pin while still ensuring the transistor is driven fully into saturation for efficient switching.

7. Block Diagram

At a functional level, the system can be represented as four interconnected blocks: Temperature Sensing -> Signal Processing (Arduino ADC + Control Logic) -> PWM Signal Generation -> Actuation (Transistor + Fan). The TMP36 continuously senses the ambient temperature and produces a proportional analog voltage. This voltage is digitized by the Arduino's internal ADC and converted into a Celsius temperature reading in software. The control logic then determines the appropriate PWM duty cycle based on pre-defined temperature thresholds. Finally, this PWM signal drives the base of the switching transistor, which in turn controls the power delivered to the fan, completing the sense-decide-act cycle. This cycle repeats continuously every 500 milliseconds, giving the system a near real-time response to changing thermal conditions.

8. Schematic Diagram

In addition to the breadboard-style Tinkercad layout, a formal schematic diagram was generated to clearly represent the electrical connections between components using standard circuit symbols. This schematic (Figure 1) shows the Arduino UNO, the TMP36 temperature sensor module, the NPN switching transistor, the current-limiting resistor, the flyback diode, and the DC fan (motor), along with their interconnections and power/ground references.

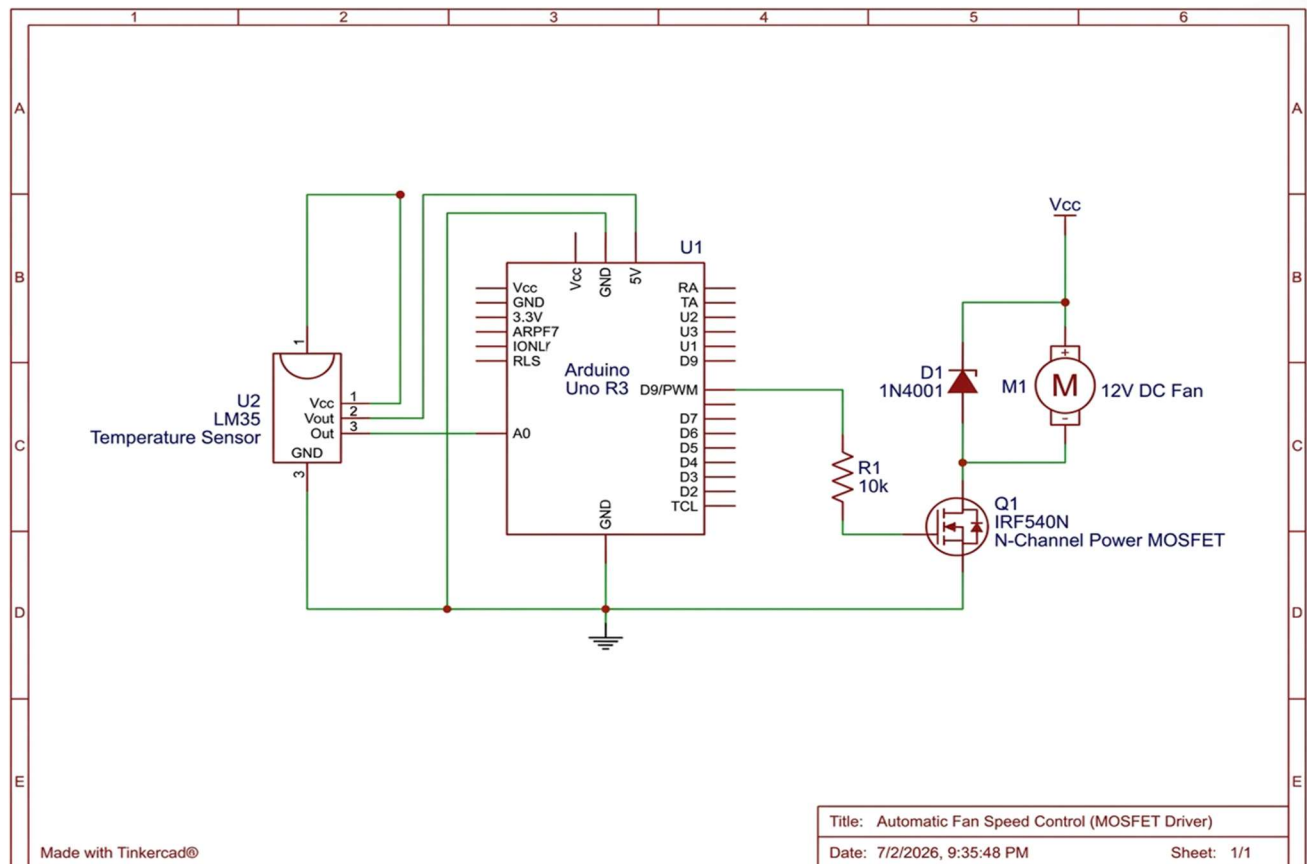
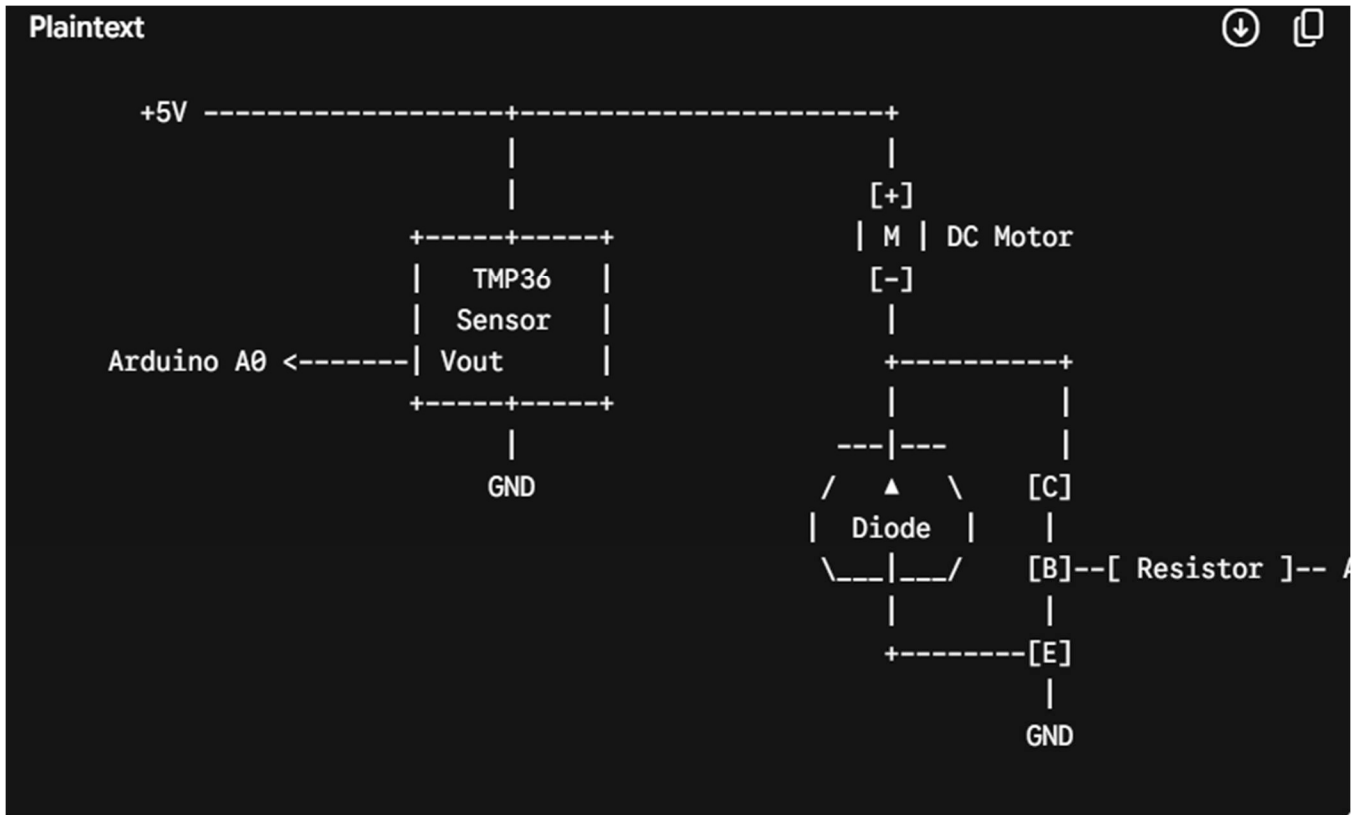


Figure 1: Schematic Diagram of the Automatic Fan Speed Control Circuit

As shown in the schematic, the TMP36 sensor (U2) draws power from the +5V rail and ground, with its analog output routed to one of the Arduino's analog input pins. The Arduino (U1) processes this reading and drives the base of transistor Q1 (NPN) through resistor R1 using a PWM output pin. The motor M1, representing the DC fan, is switched by transistor Q1, while diode D1 is connected in parallel with the motor to suppress back-EMF generated during switching. This schematic representation is useful for PCB layout, formal documentation, and communicating the design intent independently of any particular simulation tool's visual style.

9. Circuit Description

S.No	Component	Quantity	Function
1	Arduino UNO R3	1	Main microcontroller board
2	TMP36 Temperature Sensor	1	Senses ambient temperature (analog output)
3	NPN Transistor (2N2222 type)	1	Switches/drives the fan using PWM signal
4	DC Fan (Motor)	1	Cooling actuator, output device
5	Flyback Diode	1	Protects transistor from back-EMF of fan motor
6	Resistor (~1k ohm)	1	Current-limiting resistor for transistor base
7	Breadboard	1	Prototyping platform
8	Jumper Wires	As required	Circuit connections
9	USB Cable	1	Power and programming interface

The TMP36 sensor's analog output pin is connected to the Arduino's analog input A0, while its outer pins are connected to 5V and GND respectively. The sensor output voltage varies linearly with temperature. The Arduino reads this analog voltage, converts it to a temperature value in degrees Celsius, and determines an appropriate PWM value. This PWM signal is output on digital pin 9, which drives the base of an NPN transistor through a current-limiting resistor. The transistor acts as an electronic switch, allowing the fan (connected to the collector, with its other terminal to 5V) to be driven at variable speed. A flyback diode is placed across the fan terminals to protect the transistor from voltage spikes generated when the motor coil switches off.

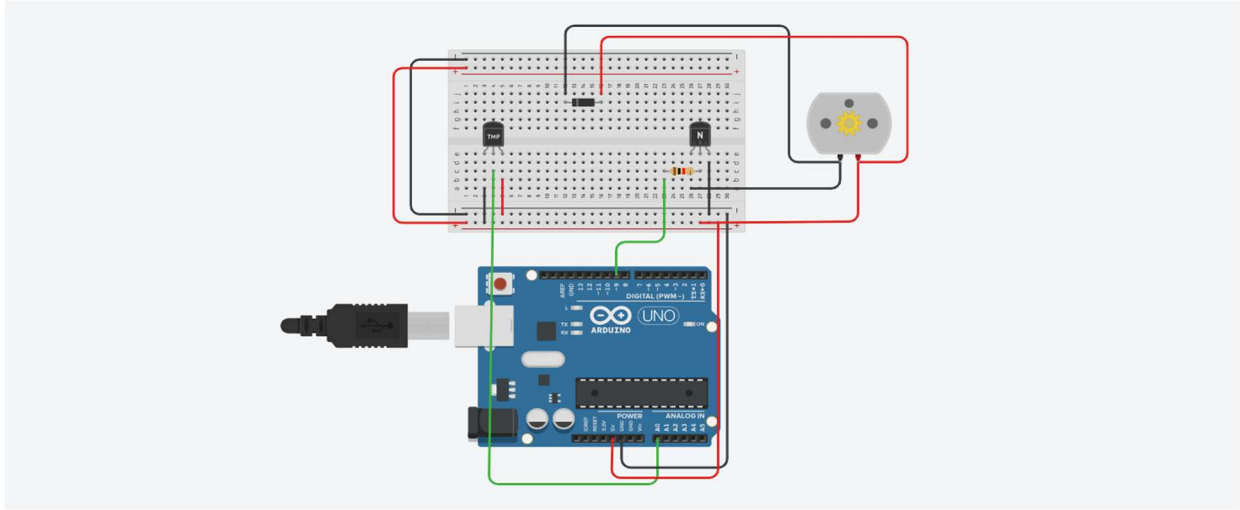


Figure 2: Circuit Layout

9.1 Pin Connection Summary

Component Pin	Connected To	Purpose
TMP36 Vcc	Arduino 5V	Sensor power supply
TMP36 GND	Arduino GND	Common ground reference
TMP36 Vout	Arduino A0	Analog temperature signal input
Transistor Base	Arduino Pin 9 (via 1k ohm resistor)	PWM control signal

Component Pin	Connected To	Purpose
Transistor Emitter	Arduino GND	Common ground / current return path
Transistor Collector	Fan negative terminal	Switched current path to fan
Fan positive terminal	Arduino 5V	Power supply to fan
Flyback Diode	Across fan terminals (cathode to +5V)	Back-EMF protection

10. Working Principle

The complete working of the system can be broken down into four sequential stages that repeat continuously in the Arduino's main loop, once every 500 milliseconds.

10.1 Stage 1 - Analog Sensing

The TMP36 continuously outputs a voltage proportional to the surrounding temperature, following the relationship: $V_{out} = 0.5V + (10mV \times \text{Temperature in degC})$. This means at 25degC, the sensor outputs approximately 0.75V, and at 0degC it outputs 0.5V. This analog voltage is fed directly to the Arduino's A0 pin.

10.2 Stage 2 - Analog-to-Digital Conversion

The Arduino UNO's internal ADC samples the voltage on A0 and converts it into a 10-bit digital value ranging from 0 to 1023, corresponding to an input voltage range of 0V to 5V (using the default reference voltage). This is performed in code using the `analogRead()` function, which returns the raw sensorValue used in subsequent calculations.

10.3 Stage 3 - Temperature Calculation and Decision Logic

The raw ADC value is converted to a temperature in Celsius using the scaling formula $\text{tempC} = \text{sensorValue} \times 0.488$. This constant is derived from combining the ADC's voltage resolution ($5V / 1024 \text{ steps} \sim 4.88mV \text{ per step}$) with the TMP36's $10mV/\text{degC}$ sensitivity, giving an approximate combined scale factor of $0.488\text{degC per ADC count}$ (a simplified approximation that omits the 500mV offset for ease of implementation, adequate for this project's relative threshold-based control). Once the temperature is known, the microcontroller compares it against two fixed thresholds - 23degC and 33degC - to select one of three PWM output levels, as detailed in the table below.

Temperature Range	Fan State	PWM Value	Duty Cycle	Approx. Voltage to Fan
Below 23 C	OFF	0	0%	0V
23 C to below 33 C	Medium Speed	128	~50%	~2.5V (avg)
33 C and above	Full Speed	255	100%	~5V (avg)

10.4 Stage 4 - PWM Actuation

The selected pwmValue (0, 128, or 255) is written to digital pin 9 using the `analogWrite()` function. Since pin 9 is one of the Arduino UNO's hardware PWM-capable pins, this generates a square wave at approximately 490Hz whose duty cycle is proportional to the pwmValue (0 = always off, 255 = always on). This PWM waveform drives the transistor's base, which rapidly switches the collector-emitter path on and off at the same frequency. Because this switching occurs much faster than the fan's mechanical response time, the fan effectively experiences an average voltage proportional to the duty cycle, resulting in smooth, proportional speed control rather than abrupt full-speed jumps. Simultaneously, the temperature and PWM values are printed to the Serial Monitor via `Serial.print()` and `Serial.println()` for real-time observation, and the loop repeats after a 500ms delay.

11. Flowchart of Operation

The logical flow of the firmware can be summarised as follows:

- START -> Initialize fanPin as OUTPUT, initialize Serial communication at 9600 baud.
- LOOP START -> Read analog value from TMP36 sensor on pin A0.

- Convert raw ADC value to temperature in degC using the scaling formula.
- DECISION: Is temperature < 23 C?
- YES: Set pwmValue = 0 (fan OFF).
- NO: DECISION: Is temperature < 33 C?
- YES: Set pwmValue = 128 (medium speed).
- NO: Set pwmValue = 255 (full speed).
- Apply pwmValue to fan pin using analogWrite().
- Print temperature and PWM value to Serial Monitor.
- Wait 500 milliseconds, then repeat from LOOP START.

12. Arduino Program

```
const int tempPin = A0;
const int fanPin = 9;
int tempC;
int pwmValue;

void setup() {
  pinMode(fanPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int sensorValue = analogRead(tempPin);
  tempC = sensorValue * 0.488;

  if (tempC < 23) {
    pwmValue = 0;
  }
  else if (tempC >= 23 && tempC < 33) {
    pwmValue = 128;
  }
  else {
    pwmValue = 255;
  }

  analogWrite(fanPin, pwmValue);
  Serial.print("Temperature: ");
  Serial.print(tempC);
  Serial.print(" C | PWM: ");
  Serial.println(pwmValue);
  delay(500);
}
```

13. Code Explanation

13.1 Global Declarations

Lines 1-4 declare the pin assignments and global variables. tempPin is set to A0, the analog input connected to the TMP36's output. fanPin is set to digital pin 9, chosen because it supports hardware PWM. The variables tempC and

pwmValue are declared globally so they can be updated each loop iteration and are accessible throughout the program.

13.2 setup() Function

The setup() function runs once when the Arduino powers on or resets. pinMode(fanPin, OUTPUT) configures pin 9 as a digital output so it can drive the transistor base. Serial.begin(9600) initializes serial communication at 9600 baud, enabling the Arduino to send temperature and PWM data to the Serial Monitor for real-time debugging and verification.

13.3 loop() Function - Sensing

analogRead(tempPin) reads the instantaneous analog voltage on A0 and returns a value between 0 and 1023 representing that voltage relative to the 0-5V reference range. This raw value is stored in sensorValue.

13.4 loop() Function - Conversion

tempC = sensorValue * 0.488 converts the raw ADC count into an approximate temperature in degrees Celsius, using the combined ADC-and-sensor scale factor described in Section 10.3.

13.5 loop() Function - Decision Logic

The if/ else if/ else conditional block compares tempC against the 23 C and 33 C thresholds and assigns pwmValue accordingly: 0 for below 23 C, 128 for the 23-33 C band, and 255 for 33 C and above. This is a simple but effective example of threshold-based control logic commonly used in embedded systems before more advanced control strategies (like PID) are introduced.

13.6 loop() Function - Actuation and Reporting

analogWrite(fanPin, pwmValue) outputs the calculated PWM duty cycle to the fan-driving transistor. The subsequent Serial.print() and Serial.println() statements format and transmit a human-readable status line (e.g., "Temperature: 25 C | PWM: 128") to the Serial Monitor. Finally, delay(500) pauses execution for half a second before the loop repeats, setting the system's overall sampling and response rate.

14. Simulation Setup

The circuit was assembled and simulated using Autodesk Tinkercad Circuits, a free browser-based platform that allows accurate simulation of Arduino-based circuits, including analog sensors, PWM-driven actuators, and real-time Serial Monitor output - making it well suited for validating embedded systems designs without requiring physical hardware. The simulated components used were: an Arduino UNO R3, a TMP36 temperature sensor block, an NPN transistor, a small DC fan actuator (which reports simulated RPM), a diode, a resistor, and a breadboard for interconnection, as shown in Figure 2. The TMP36's simulated temperature could be adjusted directly within Tinkercad to observe the system's response across a range of thermal conditions.

15. Simulation Results

The circuit was simulated in Tinkercad across a range of temperature values to verify the fan control logic. The observed behaviour is summarised below:

Simulated Temperature	Expected PWM	Observed PWM	Fan Speed (RPM)	Result
9 C	0	0	0 rpm	Pass - Fan OFF
25 C	128	128	724 rpm	Pass - Medium speed
44 C	255	255	1432 rpm	Pass - Full speed

The Serial Monitor output confirmed the expected transitions at each threshold, and the simulated fan RPM increased proportionally with the applied PWM value, validating correct operation of the control algorithm and hardware interfacing. At 9 C, well below the 23 C lower threshold, the fan correctly remained off (0 rpm), avoiding unnecessary power consumption and noise. At 25 C, within the medium band, the fan spun at 724 rpm, corresponding to the 128/255 (~50%) duty cycle. At 44 C, above the 33 C upper threshold, the fan reached 1432 rpm, close to its simulated maximum, confirming that the full PWM value (255) correctly drives maximum fan speed. The near-linear relationship observed between PWM duty cycle and resulting RPM further confirms that the transistor switching stage is operating correctly and that no signal loss or clipping is occurring between the microcontroller output and the fan.

15.1 Serial Monitor Sample Output

Representative Serial Monitor output captured during simulation at each of the three test points is reproduced below (format: Temperature reading | Corresponding PWM value):

- Temperature: 9 C | PWM: 0
- Temperature: 25 C | PWM: 128
- Temperature: 44 C | PWM: 255

16. Testing

Testing was carried out by varying the simulated ambient temperature applied to the TMP36 sensor within Tinkercad and observing three outputs simultaneously: the Serial Monitor's printed temperature and PWM values, the visual fan animation, and the fan's reported RPM figure. Three representative test points were selected to cover each of the three control zones defined in the firmware: a low-temperature point well below the 23 C threshold, a mid-range point within the 23-33 C band, and a high-temperature point above the 33 C threshold. This boundary-and-midpoint testing approach is a standard technique for validating threshold-based conditional logic, as it directly exercises each branch of the if / else if / else decision structure and confirms correct behaviour at the extremes as well as in the intermediate operating range.

17. Advantages

- Fully automatic operation - no manual switching or speed adjustment required.
- Proportional (three-tier) speed control reduces unnecessary power consumption and fan noise compared to simple ON/OFF systems.
- Low component cost and simple circuitry, using only a handful of easily available parts.
- Easily reprogrammable thresholds and PWM levels via firmware changes, without any hardware modification.
- Real-time monitoring via Serial Monitor aids debugging, calibration, and demonstration.
- Scalable design that can be extended with displays, additional sensors, or wireless connectivity.

18. Limitations

- Only three discrete speed levels are implemented; the fan does not scale continuously and smoothly with temperature.
- The temperature-to-Celsius conversion formula used ($\times 0.488$) is a simplified approximation and does not account for the TMP36's 500mV zero-offset, which may introduce a small calibration error in absolute readings.
- No hysteresis is implemented around the threshold boundaries, which could cause rapid fan speed 'chattering' if temperature hovers exactly at a boundary in a real (noisy) environment.

- The system was validated only through Tinkercad simulation in this report; physical hardware testing would confirm real-world performance.
- There is no persistent data logging; temperature history is only visible in real time via the Serial Monitor.

19. Applications

- Cooling of electronic enclosures, control panels, and server cabinets.
- Automatic thermal management in home appliances and automotive systems.
- Industrial machine cooling based on real-time temperature feedback.
- Low-cost energy-efficient HVAC-assist systems for small rooms or IoT setups.
- Educational demonstration platform for embedded systems, ADC, and PWM concepts.

20. Future Scope

- Replace fixed thresholds with a continuous PWM mapping function (e.g., `map()`) for smoother, stepless speed transitions proportional to temperature.
- Implement hysteresis around each threshold to prevent rapid fan speed oscillation when temperature hovers near a boundary.
- Add an LCD/OLED display for standalone temperature and fan-speed readout without a PC connection.
- Integrate IoT connectivity (ESP32 + cloud dashboard, MQTT, or a mobile app) for remote temperature monitoring and historical data logging.
- Add a closed-loop RPM feedback sensor (e.g., IR/hall-effect tachometer) for precise, verified speed regulation.
- Apply a proper calibration offset to the TMP36 conversion formula to account for its 500mV zero-point offset, improving accuracy.
- Replace the single-transistor drive stage with a MOSFET or a dedicated motor driver IC for higher current handling and efficiency.

21. Conclusion

The Automatic Fan Speed Control system successfully demonstrates real-time temperature sensing and proportional actuator control using embedded systems principles. By using PWM-based control instead of a simple ON/OFF relay, the system achieves smoother, more energy-efficient cooling response across three distinct operating zones. The project reinforces core embedded systems concepts including analog-to-digital conversion, conditional control logic, PWM generation, and transistor-based motor driving - all validated through hardware-in-the-loop simulation on Tinkercad. The simulation results at 9 C, 25 C, and 44 C confirmed that the system correctly identifies each temperature zone and applies the corresponding fan speed, with observed RPM values scaling appropriately with the applied PWM duty cycle. With the enhancements outlined in the Future Scope section - particularly continuous PWM mapping, hysteresis, and IoT connectivity - this design could be extended into a more robust, production-ready thermal management solution.